

Project Report

Pascal Subset Mini Compiler



Ahmad Bin Rashid (2023-CS-52)

Abubakar Munir (2023-CS-60)

Muhammad Omer (2023-CS-68)

Instructor: Ma'am Aliya

Course: Compiler Construction Lab

University of Engineering and Technology

Contents

1	Project Overview	2
2	Directory Structure	2
3	Build and Usage	2
3.1	Prerequisites	2
3.2	Commands	2
3.3	Output Files	2
4	Source Files	3
5	Component Details	3
5.1	Lexer (<code>lexer.h</code> / <code>lexer.cpp</code>)	3
5.2	Grammar Engine (<code>grammar.h</code> / <code>grammar.cpp</code>)	3
5.3	Recursive Descent Parser (<code>parser_rd.h</code> / <code>parser_rd.cpp</code>)	4
5.4	LL(1) Parser (<code>parser_ll.h</code> / <code>parser_ll.cpp</code>)	4
5.5	SLR(1) Parser (<code>parser_lr.h</code> / <code>parser_lr.cpp</code>)	4
5.6	AST (<code>ast.h</code> / <code>ast.cpp</code>)	4
5.7	Symbol Table (<code>symbol_table.h</code> / <code>symbol_table.cpp</code>)	5
5.8	Error Handler (<code>error_handler.h</code>)	5
6	Test Programs	5
7	Compiler Pipeline	6

1 Project Overview

This project implements a **multi-parser Mini Compiler** for a subset of the Pascal programming language, written in **C++17**. Given a Pascal source file (**.pas**), the compiler performs lexical analysis, grammar table construction, syntax analysis via three distinct parsers, semantic analysis, and AST generation, writing all results to an output directory.

2 Directory Structure

```
CS471L_23-CS-53_60_68/
|-- Makefile           Build automation
|-- README.md          Quick-start guide
|-- mini_compiler      Compiled binary (after make)
|-- src/               C++ source files (16 files)
|-- docs/              Technical reports and this document
|-- test/              Six Pascal test programs
'-- output/            Compiler output files
```

3 Build and Usage

3.1 Prerequisites

- g++ with C++17 support
- GNU make

3.2 Commands

```
# Compile the binary
make

# Run on a Pascal file
./mini_compiler <source_file.pas> [output_directory]

# Run all six test cases
make test

# Remove build artifacts
make clean
```

3.3 Output Files

For each input file **<name>.pas**, the following files are written to the output directory:

File	Contents
<name>_tokens.txt	Full token stream from the lexer
<name>_errors_rd.txt	Errors from Recursive Descent parser
<name>_errors_ll.txt	Errors from LL(1) parser
<name>_errors_lr.txt	Errors from SLR(1) parser
<name>_ast.txt	Abstract Syntax Tree (on successful RD parse)
<name>_symtable.txt	Symbol table history (scopes and symbols)
<name>_lr_trace.txt	Step-by-step SLR(1) shift-reduce trace
docs/parser_tables.md	FIRST/FOLLOW sets, LL(1) & SLR(1) tables

4 Source Files

File(s)	Component	Role
main.cpp	Driver	Orchestrates all compiler phases
lexer.h / lexer.cpp	Lexer	Double-buffered tokeniser
grammar.h / grammar.cpp	Grammar Engine	FIRST, FOLLOW, LL(1), SLR(1) tables
parser_rd.h / .cpp	RD Parser	Recursive descent + semantic analysis + AST
parser_ll.h / .cpp	LL(1) Parser	Table-driven predictive stack parser
parser_lr.h / .cpp	SLR(1) Parser	Shift-reduce parser with trace output
ast.h / ast.cpp	AST	Node definitions and pretty-printer
symbol_table.h / .cpp	Symbol Table	Scoped symbol manager
error_handler.h	Error Handler	Unified error collection and reporting

5 Component Details

5.1 Lexer (lexer.h / lexer.cpp)

The lexer reads the source file using a **double-buffered** scheme (two 4096-byte buffers) for efficient character scanning. It recognises:

- **Keywords:** program, const, var, procedure, function, begin, end, if, then, else, while, do, for, to, writeln, write, readln, read, div, mod, and, or, not, true, false, integer, real, boolean, char, array, of
- **Identifiers and Literals:** identifiers, integer numbers, real numbers, string literals, character literals
- **Operators:** + - * / = <> < <= > >= := : ; , . [] () ..
- **Comments:** { ... } and (* ...*)

Keywords are matched case-insensitively. Lexical errors (e.g. unterminated strings, unknown characters) are reported with line and column.

5.2 Grammar Engine (grammar.h / grammar.cpp)

The grammar engine encodes the Pascal subset grammar as BNF productions and automatically builds:

1. **FIRST sets** — iteratively computed for all terminals and non-terminals
2. **FOLLOW sets** — computed using the iterative algorithm with the \$ sentinel
3. **LL(1) Parsing Table** — maps (NonTerminal, Lookahead) to a production index
4. **SLR(1) Action & Goto Tables** — built via LR(0) item set closure and GOTO construction, with shift preferred over reduce for dangling-else ambiguity

All tables are saved to `docs/parser_tables.md` on every run.

5.3 Recursive Descent Parser (`parser_rd.h` / `parser_rd.cpp`)

A hand-written, predictive recursive descent parser. Each grammar non-terminal has a corresponding `parse*()` method. It is the only parser that:

- Builds the **Abstract Syntax Tree (AST)**
- Performs **semantic analysis** (via the Symbol Table)

Semantic checks performed:

- **Duplicate declarations** — constants, variables, procedures, functions, parameters
- **Undeclared identifiers** — all identifier usages are looked up in scope
- **Assignment to constants** — flagged as a semantic error
- **Non-callable identifiers called as procedures/functions** — detected
- **Parameter count mismatch** — checked at call sites
- **Array access on non-array identifiers** — detected

5.4 LL(1) Parser (`parser_ll.h` / `parser_ll.cpp`)

A table-driven, non-recursive LL(1) parser using an explicit symbol stack. On a table miss it invokes **panic-mode error recovery**: it scans forward in the token stream until a symbol in the FOLLOW set of the current non-terminal is found.

5.5 SLR(1) Parser (`parser_lr.h` / `parser_lr.cpp`)

A standard shift-reduce SLR(1) parser using parallel state and symbol stacks. It logs every step (symbol stack, state stack, lookahead, action) to a trace file. On error it attempts **synchronisation-state recovery** by searching the state stack for a state that has a valid GOTO on a synchronisation non-terminal (`Stmt`, `DeclPart`, etc.).

5.6 AST (`ast.h` / `ast.cpp`)

The AST is built only by the Recursive Descent parser. Node types:

Node	Represents
ProgramNode	Whole program
VarDeclNode	Variable declaration
ConstDeclNode	Constant declaration
ProcDeclNode	Procedure or function
AssignNode	Assignment statement
CallNode	Procedure / function call
IfNode	If-then-else statement
WhileNode	While loop
ForNode	For loop
WriteNode	Write / writeln
ReadNode	Read / readln
CompoundStmtNode	Begin...end block
BinaryOpNode	Binary operation
UnaryOpNode	Unary operation (not)
LiteralNode	Literal value
IdentifierNode	Variable or array element reference

5.7 Symbol Table (symbol_table.h / symbol_table.cpp)

Scoped symbol management using a **linked parent-chain** of **Scope** objects backed by `std::unordered_map`.

- **Scope entry / exit** — `enterScope(name)` / `exitScope()` called around every procedure/function body
- **Lookup** — walks up the parent chain (supports lexical scoping)
- **Insert** — returns `false` on duplicate (triggers semantic error)
- Each **Symbol** stores: name, kind (`variable` / `constant` / `procedure` / `function` / `array`), type, scope level, line number, array bounds, parameters, return type

5.8 Error Handler (error_handler.h)

Collects errors from all phases into a unified list of **CompilerError** structs (type, line, column, message). Printed with `printSummary()` to the appropriate output file.

6 Test Programs

File	Type	What it tests
valid_program1.pas	Valid	Constants, vars, procedure, two functions, if-else
valid_program2.pas	Valid	Array declaration, while loop, for loop
valid_program3.pas	Valid	Constants, read/write, if-else, boolean flag
err_lexical.pas	Invalid	Unknown character @, unterminated string
err_syntax.pas	Invalid	Missing semicolon, extra token in argument list
err_semantic.pas	Invalid	Duplicate variable, assign to const, undeclared id, param mismatch

7 Compiler Pipeline

1. **Grammar Initialisation** — Productions loaded, FIRST/FOLLOW/LL(1)/SLR(1) tables built
2. **Lexical Analysis** — Source tokenised; token stream written to `_tokens.txt`
3. **Recursive Descent Parse** — AST built; semantic checks performed; symbol table filled; errors written to `_errors_rd.txt`; AST written to `_ast.txt`; symbol table written to `_symtable.txt`
4. **LL(1) Parse** — Lexer reset; table-driven parse; errors written to `_errors_ll.txt`
5. **SLR(1) Parse** — Lexer reset; shift-reduce parse; trace written to `_lr_trace.txt`; errors written to `_errors_lr.txt`